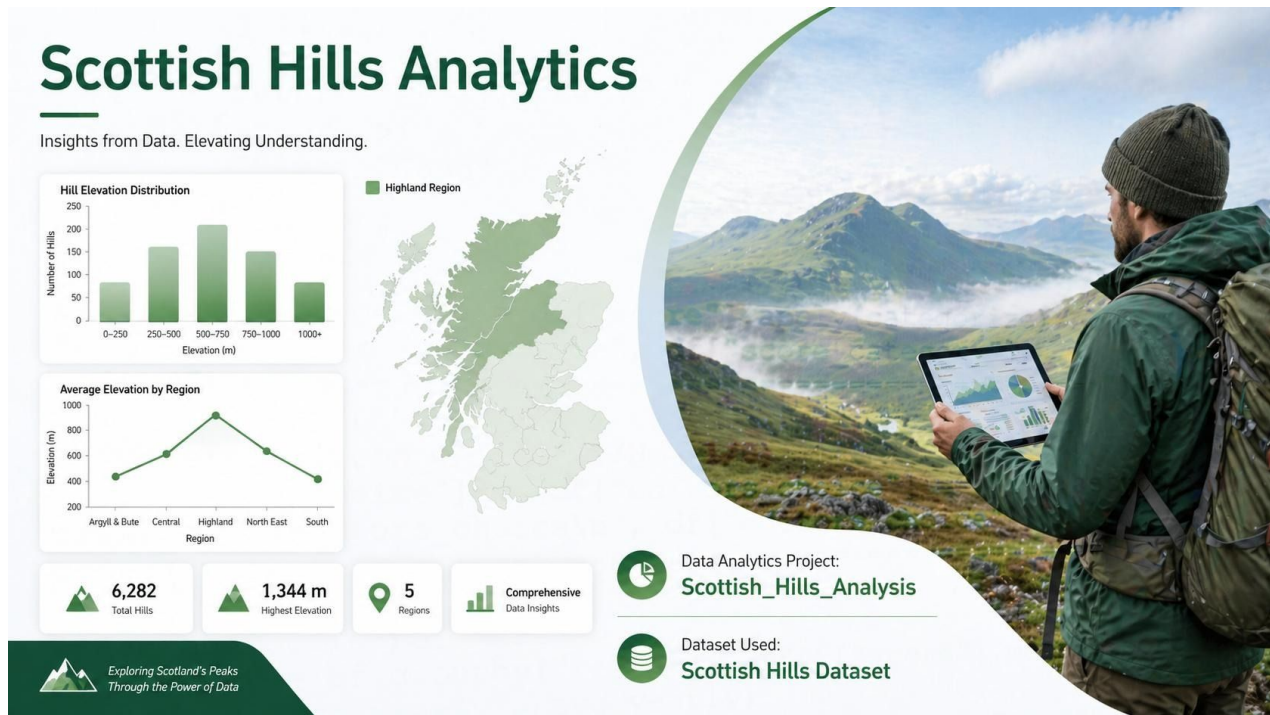




Data Analytics Project: **Scottish_Hills_Analysis**

Innovators Hub Team Members Names:

1. Neeli Ajay Kumar
2. Namratha Reddy Annam
3. Lavanya Koppula
4. Pradeep Kumar
5. Harsha Vardhan Gone

Dataset Used: scottish_hills.csv

This document outlines a structured, end-to-end data analysis project based on Scottish hills data. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

- **Dataset:** Scottish Hills Data (A CSV file)
- **Project Goal:** To analyse hill heights, regional distribution and elevation patterns over scotland over time using the scottish_hills.csv dataset.

Modules Used:

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

Project Coverage

Category	Skills Covered
Data Preprocessing	Data loading using Pandas, handling missing values (mean for Height, mode for region), type conversion (Height to numeric)
Pandas Operations	Data selection, filtering, grouping by region, aggregation and identifying top regions.
NumPy Calculations	Array conversion (Height to NumPy array)
Visualization	Line Chart, Bar Chart, Pie Chart, Saving Plots

Scenario 1: Basic Data Loading & Cleaning (Easy)

Description:

This scenario is mainly focussing on how to load the data and also cleaning Scottish hills dataset using pandas. The main goal of this scenario to display first and last rows, removing un-necessary columns, identifying missing values n important fields such as Height and Region, and applying appropriate techniques to handle them (mean for Height and mode for region). Additionally, the Height column is converted to a numeric format if it is required and also preparing the dataset for analysis. This ensures that dataset is clean and also ready for the future visualization.

Task	Description
1.	Load the dataset into a Pandas DataFrame.
2.	Display the first 5 rows and column names to understand structure.
3.	Check for missing values in Height and Region.
4.	Filling missing values (Height with mean and Region with mode)
5.	Convert Height column to numeric.

Output Screen:

- **head()** - means showing the first few rows of a dataset (by default, first 5 rows). It helps you quickly see how the data starts and understand its structure.

```
First 5 rows:
   Hill Name  Height  Latitude  Longitude  Osgrid
0  A' Bhuidheanach Bheag  936.0  56.870342  -4.199001  NN660775
1      A' Chailleach  997.0  57.693800  -5.128715  NH136714
2      A' Chailleach  929.2  57.109564  -4.179285  NH681041
3  A' Chraileag (A' Chralaig) 1120.0  57.184186  -5.154837  NH094147
4      A' Ghlas-bheinn  918.0  57.255090  -5.303687  NH008231

Column names:
Index(['Hill Name', 'Height', 'Latitude', 'Longitude', 'Osgrid'], dtype='object')

Region column not found -> Creating using Latitude & Longitude

Missing values:
Height      0
Region      0
dtype: int64

After handling missing values:
   Height  Region
0  936.0  South-East
1  997.0  North-West
2  929.2  North-East
3 1120.0  North-West
4  918.0  North-West

Data types after conversion:
Hill Name      object
Height         float64
Latitude        float64
Longitude       float64
Osgrid         object
Region         object
dtype: object
```

Conclusion:

In this scenario, the Scottish hills dataset was successfully loaded and cleaned using pandas. In the next step, we are displaying the first five rows and column names. Missing values in the Height and Region columns identified and handled using mean and mode. Additionally, the Height column was converted to a numeric data type. These steps ensure that the dataset is clean and ready for future data analysis and visualization.

Scenario 2: Line Graph + Save

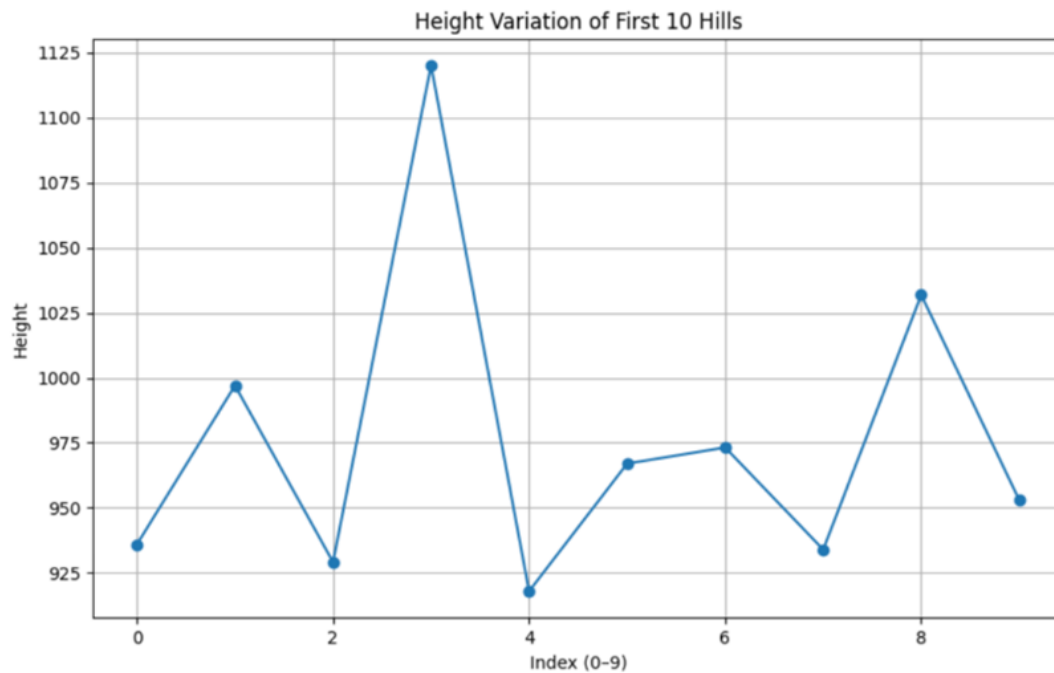
Description:

In this scenario, we analyzed the variation in hill heights using a small sample of the dataset. We selected the hill names and their corresponding heights, then focused only on the first 10 entries. The height values were converted into a NumPy array for easier handling. A line graph was plotted with the index (0–9) on the X-axis and height on the Y-axis to visualize how the heights change.

Task	Description
1.	Take Hill name and Height from the dataset.
2.	Take first 10 rows only.
3.	Convert Height into a NumPy array.
4.	Plot a line graph: ○ X-axis → index (0–9) ○ Y-axis → Height
5.	Add title and labels. Save the graph: <code>plt.savefig("hill_heights_line.png")</code>

Graph: Line Graph

- **Line graph:** The line graph is plotted with index on the X-axis and Height on the Y-axis. It shows the heights of the first 10 hills, where each dot represents one hill and the lines connecting them show which hills are taller and which are shorter,



Conclusion:

The graph clearly shows that hill heights vary from one location to another. Some hills are much taller while others are comparatively shorter. This indicates that elevation is not consistent across the dataset. The visualization makes it easy to compare heights and understand the differences quickly.

Scenario 3: Filtering + Bar Chart + Save

Description:

In this scenario, we filter all hills with a height greater than 900 meters. Then we group these tall hills by region and count how many hills are in each region. Finally, we convert the results into NumPy arrays and display them using a bar chart.

Task	Description
1.	Filter the dataset of hills where height > 900.
2.	Count the number of hills per Region.
3.	Select the top regions using Pandas.
4.	Convert the regions counts into NumPy arrays.
5.	Plot a bar chart with Region on the X-axis and count on the Y-axis.
6.	Rotate labels for clarity.

Output Screen:

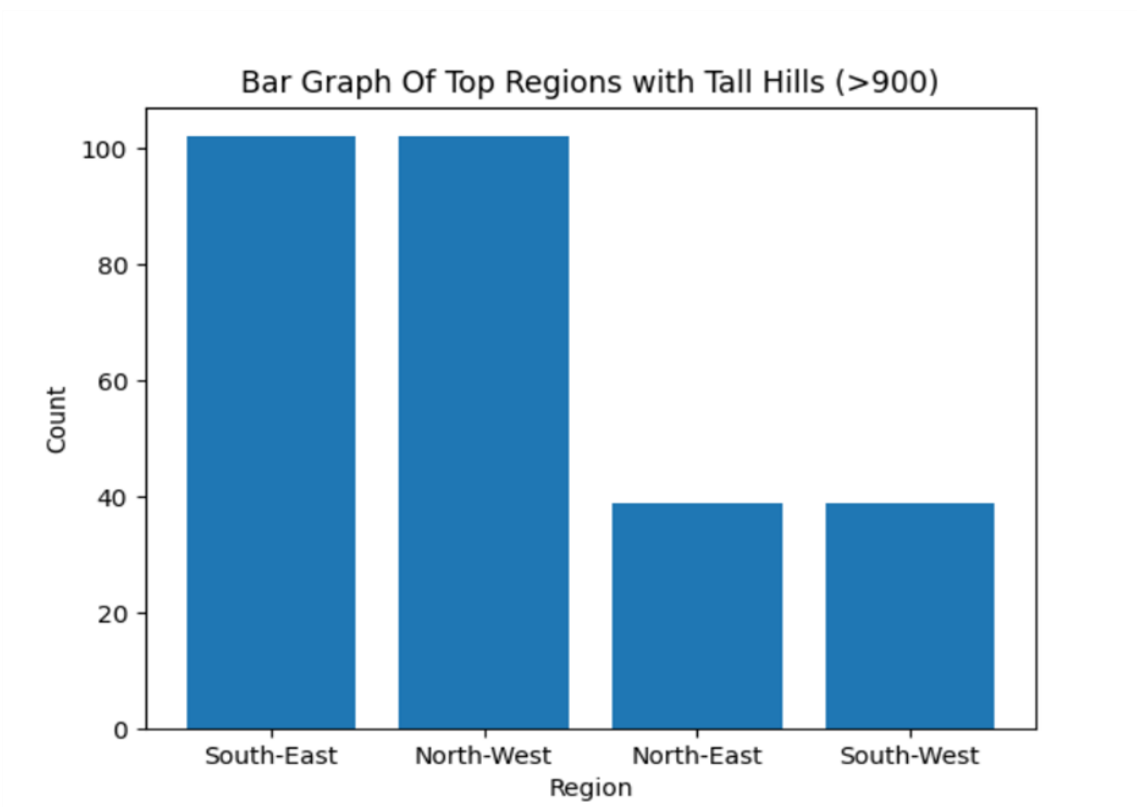
- Here, I am displaying height of hills where height is greater than 900 and also counting the region and selecting the top 5 regions

```
(S3)Tall_hills      Hill Name  Height  ...  Osgrid      Region
0      A' Bhuidheanach Bheag  936.0  ...  NN660775  South-East
1      A' Chailleach        997.0  ...  NH136714  North-West
2      A' Chailleach        929.2  ...  NH681041  North-East
3      A' Chraileag (A' Chralaig) 1120.0  ...  NH094147  North-West
4      A' Ghlas-bheinn      918.0  ...  NH008231  North-West
..      ...                ...    ...
277     The Saddle          1011.4  ...  NG936131  North-West
278     Toll Creagach       1054.0  ...  NH194282  North-West
279     Tolmount           958.0  ...  NO210800  South-East
280     Tom a' Choinich     1112.0  ...  NH164273  North-West
281     Tom Buidhe          957.0  ...  NO213787  South-East

[282 rows x 6 columns]
```

Graph: Bar Graph

- Bar Chart** - A bar chart uses rectangular bars to represent values. It is used to compare different categories, like comparing the number of high-rated games on each platform.



Conclusion:

This analysis helps us understand which regions have the most tall hills. The bar chart clearly shows the distribution of high elevation hills across different regions of Scotland

Scenario 4: Pie Chart (Region Distribution) + Save

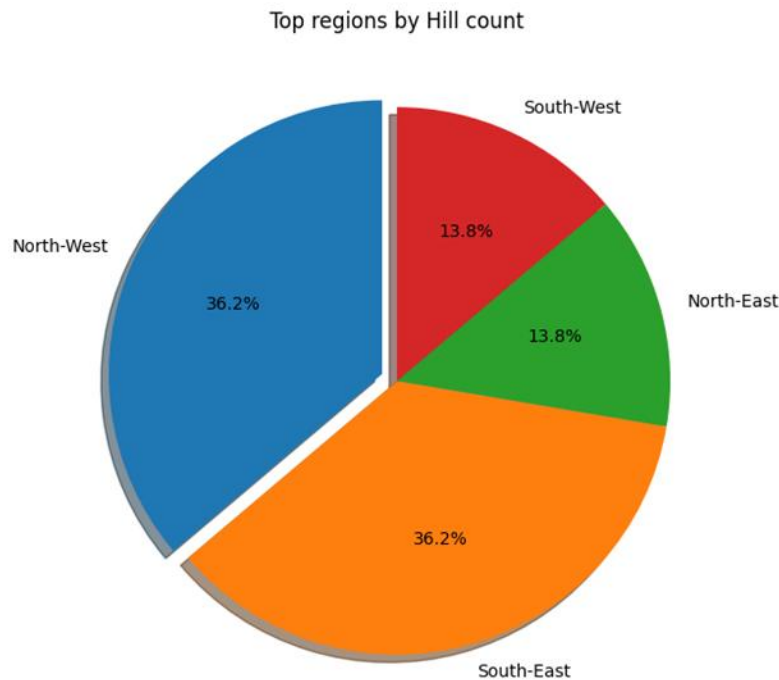
Description:

This scenario focuses on understanding how hills are geographically spread across different regions. Using Pandas, the dataset is grouped by region and the total number of hills in each area is counted. Then the top 5 regions are selected and visualized through a pie chart, where every slice represents a region along with its percentage share. This makes it easy to spot which regions have more hills and which ones have fewer.

Task	Description
1.	Count the number of Hills per Region using <code>groupby()</code> and <code>count()</code> .
2.	Select top 5 regions using <code>head(5)</code> .
3.	Prepare labels that includes region names and values that includes count of hills per region.
4.	Plot a pie chart using labels and values.
5.	Add percentage to each slice using <code>autopct='%1.1f%%'</code> .
6.	Save the graph using <code>plt.savefig("region_distribution.png")</code>

Graph: Pie Chart

- **Pie Chart:** The pie chart below, titled "Top regions by Hill count", visualizes the distribution of hills across the top 4 regions. Each slice is clearly labelled with the region name and its percentage contribution. The chart provides a clear visual comparison of how hills are distributed across regions in Scotland.



Conclusion:

The pie chart shows that how hills are spread across regions. North-West and South-East take the lead, each at 36.2%. South-West and North-East come in at 13.8% each smaller shares, but evenly matched, which shows a fairly balanced split in the remaining areas.

Overall, grouping the data by region and plotting it as a pie chart gave us a quick, clear understanding of where most hills are present

Scenario 5: Advanced Analysis + Multiple Graphs

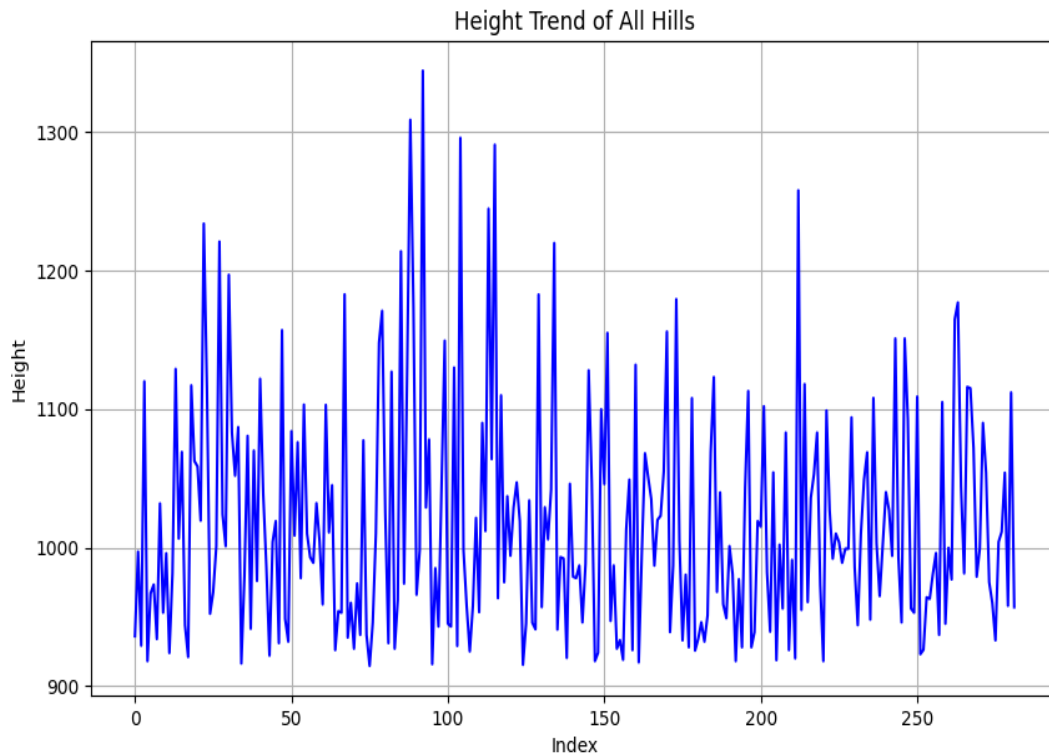
Description:

This scenario focuses on advanced analysis of hill data using feature creation, NumPy calculations, and multiple visualizations. A new column called **Height Category** is created to group hills into “Very High”, “High”, and “Moderate”. NumPy is used to calculate height differences to understand variation between hills. Different graphs like line graph, stacked bar chart, and histogram are used to visually represent the data. This helps in understanding patterns, comparisons, and distribution of hill heights.

Task	Description
1.	1. Create a new column: Height Category: , Height ≥ 1000 $\rightarrow$ "Very High" 800–999 $\rightarrow$ "High" , < 800 $\rightarrow$ "Moderate"
2.	2. Convert Height column to NumPy array. Calculate height differences using: np.diff()
3.	Plot: a) Line Graph : Plot height trend for all hills. over the years. b) A Stacked Bar Chart : Show count of Height Category per Region. C) Histogram 6. Plot distribution of Height.
4.	plt.savefig("height_trend.png") , plt.savefig("height_category_stacked.png"), plt.savefig("height_histogram.png")
5.	Insights: Which region has tallest hills , Which category is most common, Distribution pattern of heights give discription and about graph

Graph: Line Graph

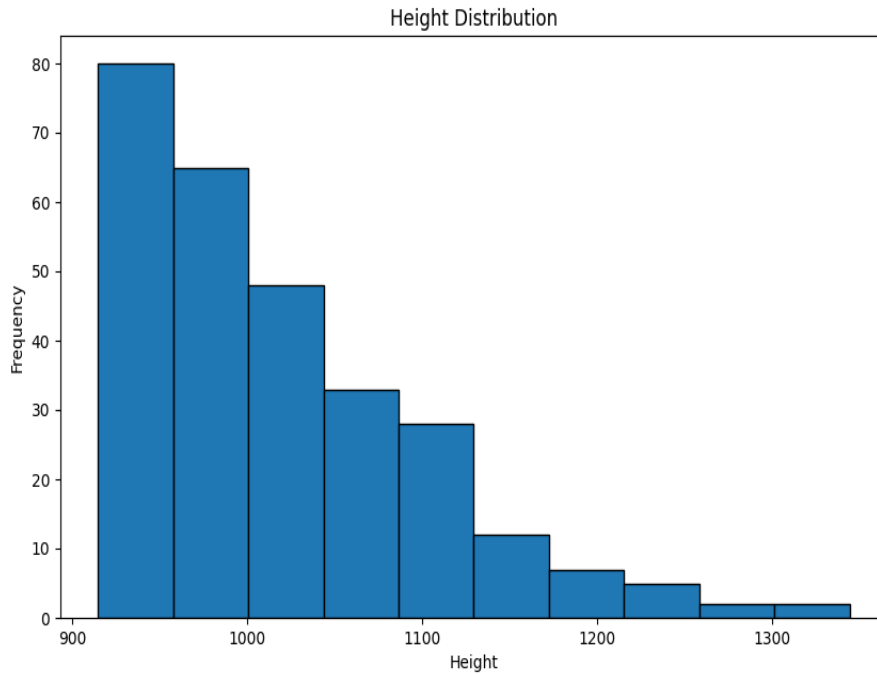
- **Line graph:** The line graph is plotted with index on the X-axis and Height on the Y-axis. It shows the heights of the first 10 hills, where each dot represents one hill and the lines connecting them show which hills are taller and which are shorter



This graph shows how the height of hills changes across the dataset.

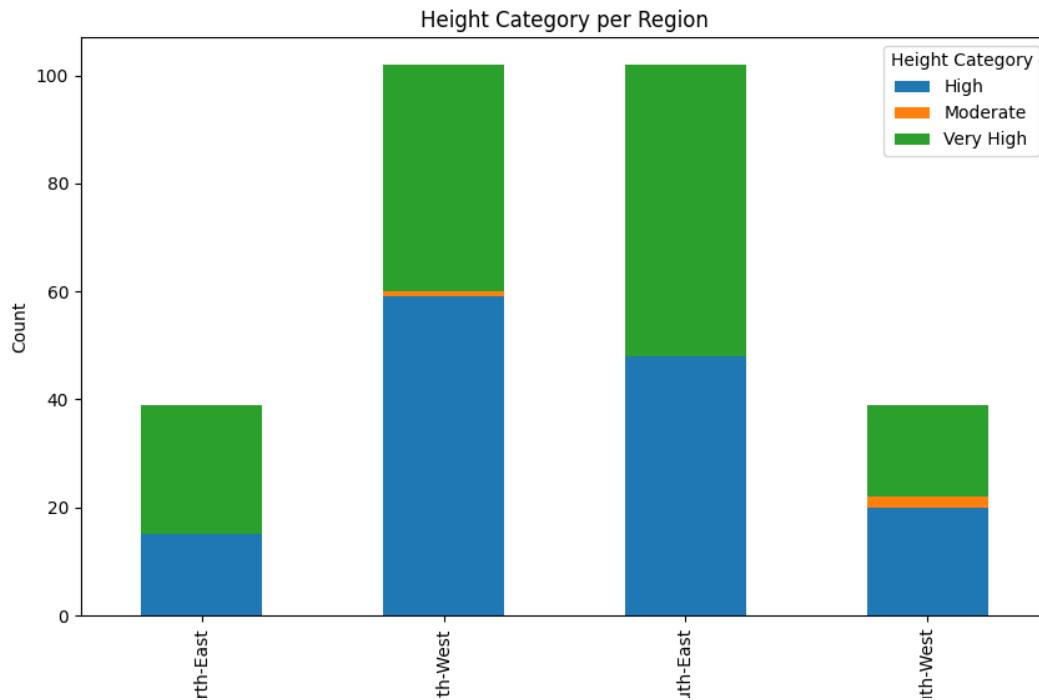
1. Heights go up and down frequently
2. There is **no smooth pattern**
3. Some peaks show very tall hills (above 1300)

- **Histogram**: A chart that shows how data is distributed by grouping values into ranges (bins) and counting how often they occur.



1. Most hills are between **900 and 1050 height**
2. Fewer hills are very tall (above 1200)
3. The graph is **right-skewed** (more smaller hills, fewer very tall hills)

- **Stacked bar chart**: A bar chart where multiple categories are stacked on top of each other to show parts of a whole.



1. Some regions have more **Very High hills**
2. Some regions have more **High hills**
3. Moderate hills are fewer compared to others

Conclusion:

This scenario helps in understanding hill data in a deeper way by combining categorization, numerical analysis, and visualization. The use of graphs makes it easier to identify patterns and compare regions. NumPy helps in analyzing height variations efficiently. Overall, this scenario provides meaningful insights about hill distribution and characteristics.

Final Conclusion:

This project covers data analysis from basic to advanced levels. It starts with data cleaning and preparation, then moves to filtering and grouping data, and finally uses visualization techniques for deeper insights. Each scenario builds on the previous one, improving understanding step by step. By the end, it provides a complete overview of how to analyze and visualize real-world datasets effectively.

What you have learned

In this project, I learned how to load and clean data using Pandas and handle missing values. I used NumPy for numerical calculations and feature engineering to create new columns. I also learned how to create different graphs like line charts, bar charts, and histograms using Matplotlib. Finally, I learned how to analyze data and extract useful insights from visualizations.